

Technical Proposal Document for



Submitted to

Mr. Kosmas Raptopoulos
Chief Innovation Officer (CIO)

Submitted By



Kevit Technologies

530, 5th Floor, Level 6, 150 Ft. Ring
Road, Rajkot, Gujarat - 360001, India

Prepared By

Shubham Shah
shubham.shah@kevit.io
+91 84859 00714

Project Scope

Implementation of ETL tool

Version

1.0

Date

December 19th, 2025

Table of Contents

1 Executive Summary	4
2 Company Profile	5
2.1 About Kevit.....	5
2.1.1 Our Core Values.....	6
2.1.2 Organization Structure	6
2.2 Core Capabilities	7
2.2.1 Data Engineering, ETL Frameworks & Data Automation	7
2.2.2 Artificial Intelligence and Business Automation	7
2.2.3 Web and Mobile Application Development.....	7
2.3 Relevant Experience	8
2.4 Client References	11
3 Scope and Deliverables	12
3.1 Scope.....	12
3.1.1 Data Ingestion Layer.....	12
3.1.2 Transformation Engine	13
3.1.3 File Processing and Aggregation Layer.....	14
3.1.4 Job Orchestration & Data Load Layer.....	14
3.2 Deliverables.....	15
3.3 Assumptions & Considerations	16
4 Technical Approach	17
4.1 Overview	17
4.2 Architecture Diagram	18
4.3 Technical Stack	20
4.4 Non-Functional Requirements Alignment	20
5 Sample Use Case Solution.....	21
6 Project Execution Plan.....	22
6.1 Implementation Phases	22
6.2 Team Allocation.....	23
6.2.1 Role-wise Allocation.....	23
6.2.3 Resource Details.....	24
6.3 Execution Methodology	25
6.3.1 Milestone Execution Methodology	25
6.3.2 Lifecycle of features	27
6.3.3 Responsibility Matrix.....	27
6.3.4 Testing and QA Approach	28

6.3.5 Team Availability	29
6.4 Change Management Process.....	29
6.4.1 Roles & Responsibilities	29
6.4.2 Change Request Submission	29
6.4.3 Initial Review & Impact Assessment	30
6.4.4 Approval & Prioritization.....	30
6.4.5 Implementation & Validation.....	30
6.4.6 Documentation & Traceability	30
6.5 Training and Knowledge Transfer Plan	31
6.6 Risk Management	31
6.6.1 Risk Assessment Scale.....	31
6.6.2 Risk Register	32

1 Executive Summary

EXUS operates in a highly heterogeneous client data environment where large volumes of structured data are received from multiple external systems, each following different formats, schemas, encodings, and delivery schedules. Reliable data ingestion, schema normalization across approximately 22 target tables, and the ability to process very large files within tight operational windows are business critical. Existing approaches based on custom scripts and manual intervention introduce scalability limits, operational risk, and long term maintenance overhead as client volume continues to grow.

Kevit proposes a configuration driven, containerized ETL platform designed specifically to standardize and automate EXUS client data onboarding. The solution ingests text-based files from SFTP or cloud storage, applies configurable validation, transformation, and aggregation rules, and loads normalized data into EXUS databases using high performance bulk operations. All behaviour is defined through version controlled YAML configuration files, eliminating per client code customisation while allowing rapid onboarding of new sources, schemas, and transformation logic.

The proposed architecture is intentionally low risk and future-proof. It is built entirely on open source technologies aligned with EXUS's existing ecosystem, including Python, Apache Airflow, SQL based processing, Docker, and Kubernetes. The platform follows stateless, configuration first principles, supports GitOps workflows, and is designed to handle files containing up to 100 million records through streaming, chunked processing, and isolated Kubernetes worker execution. Full source code ownership, documentation, and deployment artifacts are transferred to EXUS, ensuring zero vendor lock in.

Key Benefits Aligned to EXUS Challenges

EXUS' Challenges	Kevit's Proposed Solution
Highly variable client file formats	Flexible ingestion engine supporting delimited and fixed length files, multiple encodings, and configurable parsing rules
Schema misalignment across 22 tables	Configuration driven transformations and aggregation datasets to normalize data consistently
Manual effort and custom scripts	YAML based configuration model with no per client code changes
Performance constraints on large datasets	Streaming processing, DuckDB vectorized execution, and database bulk loaders
Limited operational visibility	Centralized orchestration, logging, retries, and alerting via Airflow
Scalability as client volume grows	Kubernetes based execution with horizontal scaling and resource isolation

Kevit is uniquely qualified to deliver this solution due to its proven experience building large scale, configuration driven ETL platforms handling heterogeneous client data, high data volumes, and production grade operational requirements similar to those faced by EXUS.

2 Company Profile

2.1 About Kevit

Legal Entity: Kevit Infotech LLP

Founded: 2019

Company Size: 130+ Professionals

Headquarter: Gujarat, India

Projects Completed: 130+

The foundation of our company dates back to a simple yet powerful idea: technology should remove complexity, not create it.

The story began when two university professors, Amit and Hemang, driven by a shared vision to bring simplicity, intelligence, and automation into businesses, started experimenting with innovative ways to solve real-world operational challenges. Their passion for applying computational thinking to business pain points inspired two of their students, Dhruvan and Harpal, who shared the same curiosity and ambition.

Together, the four of them transformed a research-driven initiative into a real organisation.

We are serving clients across industries and geographies. The company today stands at the intersection of engineering, automation, and AI, helping enterprises and emerging businesses embrace the future of digital transformation.



Business Verticals:

- Data Solutions
 - Website: <https://rapidminers.in>
 - USP:
 - Data extraction & Processing
 - Data Pipeline Management
 - Data analytics & visualization
- IT Software Service
 - Website: <https://kevit.io>
 - USP: SaaS Application, Secure & Scalable Architecture, Agentic AI Development & Consultation
- Customer Engagement Platform
 - Website: <https://chatomate.in/>
 - USP: Customer Acquisition, Retention & Support

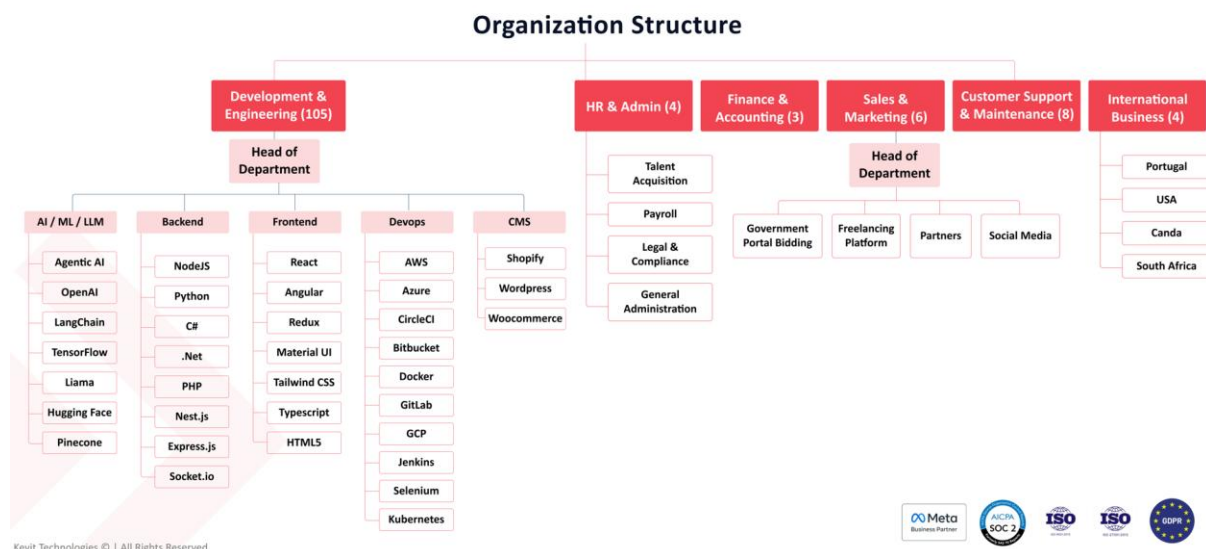
2.1.1 Our Core Values

At Kevit, our core values are the foundation of every project we deliver. They reflect how we think, operate, and build trust - as a long-term technology partner committed to solving real problems, creating value, and growing with our clients.

- Continuous Learning & Training
- Engineering excellence with AI adoption
- Client-Centered Thinking
- Higher accountability and ownership
- Customer success mindset

2.1.2 Organization Structure

Below is the structure outlining Kevit's functional teams, leadership alignment, and core capability areas. This structure demonstrates how responsibilities are distributed across engineering, operations, business, and support functions to ensure scalable delivery, clear ownership, and effective collaboration for project execution.



2.2 Core Capabilities

2.2.1 Data Engineering, ETL Frameworks & Data Automation

Data engineering and ETL platforms are a primary capability at Kevit. We design and implement configuration driven ETL frameworks that ingest text based data from multiple external systems, apply standardized validation and transformation rules, and load normalized data into relational database schemas.

Our capabilities directly align with EXUS's functional and non functional requirements and include:

- Configuration driven ETL frameworks with no per source code customization
- High volume file ingestion from SFTP and cloud storage
- Support for delimited and fixed length file formats with multiple encodings
- Schema mapping, validation, and normalization across multi table database schemas
- Reusable transformation and aggregation logic defined through configuration
- Automated job orchestration, scheduling, and dependency management
- Operational logging, monitoring, retry handling, and failure visibility

We also provide data visualisation and reporting solutions, enabling leadership teams to monitor KPIs, pipeline health, and operational insights through dashboards.

2.2.2 Artificial Intelligence and Business Automation

Kevit also delivers automation and intelligent processing capabilities that complement data platforms and operational workflows. We also build conversational AI solutions to automate Customer Acquisition, customer engagement and customer support over multiple communication channels like WhatsApp.

Our expertise:

- Building customer engagement solutions with AI
- Workflow automations
- Agentic AI – Live chat automation
- Conversational AI (Chatbots and Virtual Assistants) across various platforms such as WhatsApp, Website, Facebook, Instagram, Telegram etc.

While not central to the current scope, these capabilities support future automation and operational efficiency initiatives.

2.2.3 Web and Mobile Application Development

We develop modern, scalable digital platforms tailored to enterprise and SME's operations.

Our expertise includes:

- Application Development (Customer portal and Admin portals)
- APIs & Microservices (Monolithic & Microservices)
- Workflow management platforms
- Cloud-native architecture & microservice

2.3 Relevant Experience

Kevit has delivered ETL & data processing platforms that closely mirror EXUS's operating model, particularly in areas of heterogeneous source data, configuration-driven processing, reliability, & scale.

1. Kinetiq

Enterprise Media Intelligence Platform

Client: Kinetiq TV (<https://kinetiq.tv/>)

Region: United States

Engagement Duration: 4+ Years and ongoing

Industry: Media, Broadcasting, Advertising & Digital Marketing

Contact Person: Rishit Shah, CTO

Email: rshah@kinetiq.tv

Pain Points

Kinetiq processes large volumes of unstructured data from multiple global media and social platforms. Increasing data volume and velocity made reliable high frequency ingestion, data consistency, and operational visibility difficult, while the absence of a unified ingestion and normalization framework limited standardization, monitoring, and scalable processing without manual intervention.

Solution Provided

Kevit delivered a production grade, microservices based ETL platform supporting asynchronous, high frequency ingestion at scale. Centralized validation and normalization standardized heterogeneous inputs into a unified schema, while isolated parallel processing, incremental refresh pipelines, and end to end observability ensured fault tolerance, scalability, and reliable delivery of analytics ready data with minimal operational overhead.

Impact

- Processed ~30,000+ records per day across multiple platforms
- Executed 8 ETL cycles daily with full automation
- Enabled concurrent ingestion from 6 global data sources
- Delivered consistent, analytics-ready data despite unstructured inputs

Case Study

[Click here to view](#) (The case study provides an in-depth overview of business challenges, technical approach, snapshots of the system and measurable business impact.)

Testimonial

"The Kevit team quickly grasped the complexity of our data pipelines and the scale at which we operate. They didn't just execute requirements-they actively collaborated with us, challenged assumptions where needed, and recommended solutions that were both scalable and future-proof. Their proactive communication, technical depth, and ownership mindset gave us confidence throughout the project. The ETL platform they delivered has significantly improved the reliability and performance of our data processing and continues to support our growth seamlessly."

– **Rishit Shah**, CTO at Kinetiq TV

[Click here to view Video Testimonial](#)

2. AiCollect

Data Commercialisation Platform

Client: Rockside Consults (<https://rocksideconsult.com/>)

Region: Uganda

Engagement Duration: 11+ Months and ongoing

Industry: Agri-Tech

Contact Person: Edward Male, CEO

Email: infodnmc@gmail.com

Pain Points

Rockside operates in a data-centric environment where large volumes of structured and unstructured data are collected through customizable digital forms used by field teams and internal operations. As the platform scaled, AICL faced significant challenges ingesting historical and offline data stored in Excel files and other unstructured formats.

Manual data entry processes were slow, error-prone, and difficult to scale. Inconsistent data quality, lack of validation, and schema mismatches made it challenging to reliably onboard legacy datasets and ensure downstream data integrity.

Solution Provided

Kevit designed and implemented a robust, validation-driven ETL ingestion engine that enabled bulk data imports through dynamically generated Excel templates mapped directly to internal schemas.

The solution introduced a multi-layer validation and normalization pipeline, ensuring that only fully compliant data was allowed into the system. Entire files were automatically rejected when validation rules failed, with precise row-level and field-level error feedback provided to users, eliminating manual guesswork and rework.

Impact

- Reduced manual data entry effort by ~50%
- Improved data accuracy and consistency by ~40%
- Enabled seamless onboarding of large historical datasets
- Eliminated invalid data from entering downstream systems

Case Study

[Click here to view](#) (The case study provides an in-depth overview of business challenges, technical approach, snapshots of the system and measurable business impact.)

Testimonial

"Kevit is like a saviour for us when we were struggling to find tech providers. They invested time in deeply understanding our data challenges and operational goals and were consistently proactive in proposing the most effective and scalable approaches. The ETL architecture they delivered is robust, high-performing, and has become a critical foundation of our data operations. We truly valued their partnership and problem-solving mindset throughout the engagement."

– **Edward Male, CEO at Rockside Consults**

[Click here to view Video Testimonial](#)

3. 7Park Data

Multi-Vendor ETL Stabilisation and Data Pipeline Modernisation

Client Name: AlternativeData (<https://alternativedata.org/> - Formerly 7Park Data)

Region: United States

Engagement Duration: 18 Months

Industry: Finance (Institutional Investors & Hedge Funds)

Contact Person: Rupesh Malladi, VP Engineering

Email: NA (As he has changed the organisation and currently working at Fortegra)

Pain Points

- Legacy ETL architecture could not handle growing data volumes, causing frequent pipeline instability.
- Late, missing, or malformed vendor data triggered cascading failures and operational risk.
- Limited automation and monitoring increased manual intervention and delayed issue detection.
- Data freshness issues impacted downstream consumers.

Solutions Provided

Kevit redesigned the ingestion, orchestration, and processing layers to stabilize a multi vendor ETL ecosystem. A standardized, vendor aware ingestion framework with automated validation ensured reliable intake despite irregular vendor deliveries. Distributed Spark pipelines enabled efficient processing of tens of millions of records, while sensor driven validation and dependency management prevented cascading failures.

Impact

- Reduced data pipeline failures and delayed report deliveries by ~40% through fault-tolerant DAG orchestration and automated validation.
- Decreased vendor dependency-related breakdowns by ~15%, minimizing cascading failures caused by late or missing data.
- Lowered manual operational intervention by ~35% via automated monitoring, alerts, and vendor readiness notifications.
- Accelerated issue detection and resolution cycles, enabling faster recovery from ingestion or processing anomalies.
- Established a scalable and resilient ETL foundation, supporting future vendor onboarding with reduced operational risk.

Case Study

[Click here to view](#) (The case study provides an in-depth overview of business challenges, technical approach, snapshots of the system and measurable business impact.)

Testimonial

"Working with the Kevit team was a seamless experience. They took the time to truly understand our challenges and business goals, and consistently proposed practical, scalable solutions. The outcome has been reliable, high-quality, and has become an important part of our operations. We genuinely valued their work and proactive problem-solving approach throughout the engagement."

– **Rupesh Malladi**, VP Engineering at 7Park Data

2.4 Client References

Afrika Financial Group (<https://afrikabanque.sn/>)

Region: Senegal & Côte d'Ivoire (West Africa)

Industry: Banking & Financial Services

Projects:

1. WhatsApp Chatbot: AI-driven banking assistants developed for Banque de Dakar (Senegal) and Banque de Abidjan (Côte d'Ivoire) to improve customer banking experience.
2. AI Credit Analysis System: Developing an AI driven credit analysis platform to support the bank in assessing loan applications and managing credit risk using multiple data sources and applicant financial profiles.
3. Mobile Application: Developed a secure mobile wallet application for Galactique, enabling digital payments and card based services through an exclusive collaboration between Banque Afrika and Real Madrid Football Club.

Contact Person: Francisco Almeida, Managing Director

Email: francisco.almeida@groupeafrika.com

FlowTribe (<https://www.flowtribe.io/> - Acquired by Visma Group)

Region: Belgium, Europe

Industry: Legal and Accounting

Projects:

1. Accounton.io: Cloud-based platform streamlining tax filing and accounting workflows for professionals.
2. Lawren.io: Legal tech solution offering personalized chatbots to automate legal and compliance processes.
3. AMLCare: An anti-money laundering tool assisting accountants in meeting compliance requirements efficiently.
4. Kamigo: Client onboarding and case management portal designed for accountants to enhance productivity.

Contact Person: Dominic Wijnen, Co-founder

Email: dominic@accounton.io

3 Scope and Deliverables

3.1 Scope

The ETL solution should provide a comprehensive, configuration-driven data integration platform that transforms source files into structured database records through automated ingestion, transformation, aggregation, and loading.

- Data Ingestion Layer
- Transformation Engine
- File Processing & Aggregation
- Database Integration & Orchestration

3.1.1 Data Ingestion Layer

The Data Ingestion Layer is responsible for securely receiving raw data files from EXUS client systems and preparing them for processing. Since client data arrives in multiple formats, encodings, and structures, this layer acts as the first normalisation checkpoint.

It ensures that files - regardless of how they are structured - can be reliably read, interpreted, validated, and passed to the transformation engine in a consistent manner. This layer also handles file-level validations, filtering, and error controls to prevent incorrect or incomplete data from entering downstream systems

Implementation

RFP Requirement	Implementation Details
Text Qualifiers	Support for enclosing characters (e.g., "Text value") with a configurable qualifier character
File Encoding	ANSI, UTF-8, UTF-16 with or without BOM support
Column Headers	Support for files with or without headers Configurable header row detection
Skip Rows	Ability to skip first N rows (configurable per file)
Comment Lines	Define the comment character per file Lines starting with a comment character are ignored
Line Endings	Recognize \n, \r\n, \r line endings
Filename Regex	Support regex patterns for date stamps in filenames (e.g., filename_YYDDMM hh:mm:ss.txt)
Bad Records Threshold	Configurable threshold Raise an error when the threshold is reached
Empty File Behavior	Configurable action: continue or raise an error
Row Filtering	Filter rows based on column values (e.g., Column1.value > 100)
Delimited Files	Comma, pipe, semicolon, tab delimiters Escape character handling
Fixed-Length Files	Position-based column extraction using start/end positions
SFTP Integration	Secure connection to client SFTP servers for file ingestion

3.1.2 Transformation Engine

The Transformation Engine is the core intelligence layer of the ETL solution. It converts raw, unstructured client data into clean, standardized, and business-ready data that aligns with EXUS's internal schema and operational needs.

This layer applies configurable rules to transform column values, clean inconsistencies, enrich data, and derive new fields - without requiring code changes. All transformation logic is driven through configuration, ensuring flexibility and reusability across clients.

Implementation

RFP Requirement	Implementation Details
Type Casting	String to integer, decimal, boolean, date conversion
Date Parsing	Multi-format date parsing based on ISO standards
Trim Operations	Trim left, right, or both
Substring	Extract part of a column (substring operations)
Case Conversion	Convert to uppercase/lowercase
Regex	Apply regex patterns for matching and replacement
Remove non-printable	Removal of non-printable characters from fields
Null/Empty Handling	Replace null/empty values with default values
Rounding	Numeric rounding capability
Calculated Columns	Support for:
	<i>Fixed values</i>
	<i>Arithmetic functions (+, -, *, /)</i>
	<i>Date functions (extract day, day of week, month, year)</i>
	<i>If-then-else statements</i>
	<i>Column concatenation</i>
	<i>Autogenerated keys (UUID or sequence)</i>
	<i>Function combinations (e.g., if cast(trim(column1), integer) > 5) then '0' else column2)</i>

3.1.3 File Processing and Aggregation Layer

This layer enables data consolidation and summarisation. It allows EXUS to merge multiple data sources, group related records and generate aggregated datasets that match downstream database structures.

The aggregation layer ensures that derived datasets retain transformation logic consistency while allowing EXUS to extract meaningful insights such as customer-level, product-level, or account-level summaries from transactional data.

Implementation

RFP Requirement	Implementation Details
Merge Files	Ability to merge multiple files into one
Aggregation	Group by columns with aggregation functions
Aggregation Functions	Min, Max, First, Last, Count, Sum, Avg
Transformation Inheritance	Aggregated files have the same column transformation properties as source files (where applicable)
Test Run	Ability to test run on any file to verify transformations before actual execution

3.1.4 Job Orchestration & Data Load Layer

The Job Orchestration layer controls how and when data flows through the system. It defines execution order, dependencies, error handling, and final data loading into EXUS's database schema.

This layer ensures that ingestion, transformation, aggregation, and database operations happen in the correct sequence and under controlled conditions. It also enables automation, monitoring, and recoverability for large-scale data operations.

Implementation

RFP Requirement	Implementation Details
Copy Data	Copy data from files to database tables
Copy Modes	Insert/Update (Upsert): Insert if new, update if exists
	Update Only: Update existing records only, skip non-existent
Update Behaviour	Define update behaviour when the source is empty:
	<i>Update the field with an empty value</i>
	<i>Leave existing value as is</i>
Column Mapping	Configurable mapping from source columns to destination database columns
Delete Data	Ability to delete data from database schema

3.2 Deliverables

Based on our understanding of the RFP requirements, the following components are required to be delivered:

Deliverables	Description / Capabilities
Core ETL Engine	Configuration-driven file processing engine supporting delimited and fixed-length file formats
	Multi-encoding support (UTF-8, UTF-16, ANSI) with automatic encoding detection
	Flexible parsing engine with header detection, row skipping, and comment line handling
	Bad record isolation with configurable threshold limits
Transformation Framework	30+ built-in transformation functions including type casting, string operations, and date functions
	Calculated column engine supporting arithmetic operations, conditional logic, and function combinations
	Handling of null and empty values with default value support
	Regex-based pattern matching and replacement capabilities
Data Processing Capabilities	File merging and union operations across multiple data sources
	Group-by aggregation engine with 7 aggregate functions: MIN, MAX, SUM, AVG, COUNT, FIRST, LAST
	Deduplication logic to normalize denormalized source data
	Test run capability for validation in pre-production environments
Database Integration	Insert/Update (Upsert) support with configurable join keys
	Column-level update control with empty value handling options
	Bulk loading optimizations for high-performance data insertion
	Data deletion support with conditional delete and truncate modes
Workflow Orchestration	Apache Airflow integration for orchestration and scheduling
	File arrival detection with configurable timeout and retry logic
	Task sequencing with dependency management
	SQL execution capabilities for post-processing tasks
	Centralized error handling with configurable failure behaviours
Configuration & Deployment	YAML-based configuration files for all components
	Configuration validation framework to prevent runtime errors
	Deployment documentation and operational guides

3.3 Assumptions & Considerations

- EXUS Team Availability
 - EXUS stakeholders available for requirements workshops
 - EXUS technical team available for architecture reviews and weekly updates
- Data & Source Systems
 - Sample data files from all source systems to be provided
 - File format specifications (delimiters, encoding, schema) documented and accurate
 - Source systems produce files in a consistent format (no frequent schema changes)
 - All 22 target tables schema finalised before Phase 2 starts
- Deployment & Operations
 - EXUS DevOps team handles all containerization (Docker image builds), infrastructure provisioning and CI/CD pipelines and performs actual production deployment.

We provide source code, documentation, and deployment support/guidance.

4 Technical Approach

4.1 Overview

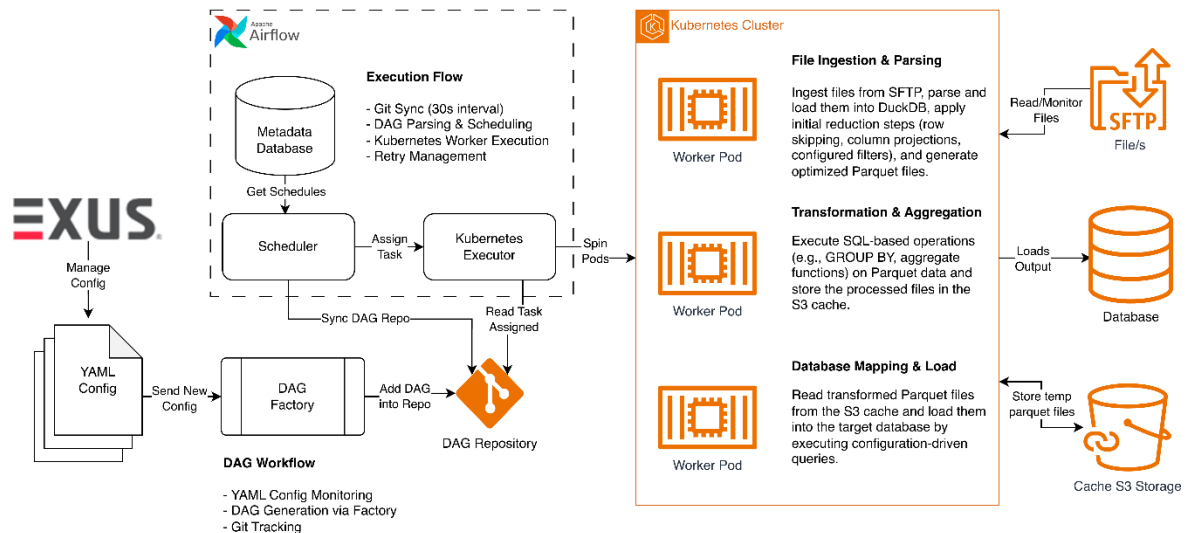
Our ETL solution follows a configuration-driven architecture where the entire pipeline operates through YAML configuration files without code customization, enabling rapid deployment and easy maintenance.

Process Flow

- Ingestion: Files arrive via SFTP (or shared/object storage). The ingestion layer monitors for file arrival, validates file presence, and triggers the pipeline. Support for multiple file formats (delimited, fixed-length), encodings (UTF-8, UTF-16, ANSI), and quality checks (bad record thresholds, empty file handling) ensures robust intake.
- Parsing: Structure extraction (delimiter/fixed-position), text qualifier handling, bad record isolation
- Transformation: Type casting, string operations, calculated columns using 30+ built-in functions plus embedded Python scripts
- Aggregation: Group-by operations with aggregate functions (MIN, MAX, SUM, AVG, COUNT, FIRST, LAST) for normalization
- Loading: Bulk database operations (insert/update/upsert) using native mechanisms (Oracle SQL*Loader)
- Orchestration:
 - Apache Airflow orchestrates the workflow using a DAG Factory pattern. Each YAML job configuration generates an Airflow DAG (1 config = 1 DAG); git-sync keeps the scheduler updated on short intervals.
 - Airflow Scheduler enforces dependencies/retries; the Kubernetes Executor spawns isolated worker pods per task.
 - Jobs cover file sensors, parse/transform/aggregate tasks, and load tasks. Data processing uses DuckDB for vectorized performance across scales (thousands to 100M+ records) without distributed overhead. Airflow handles wait-for-file, timeouts, SQL post-tasks, and retries.

4.2 Architecture Diagram

Our ETL solution is GitOps-driven configuration changes generate Airflow DAGs that orchestrate isolated Kubernetes worker pods, with intermediate Parquet stored in shared object storage (e.g., S3-compatible) between tasks. This matches the provided architecture diagram.



Architecture Overview

- **GitOps Workflow (DAG Factory)**
 - Developer commits YAML job configuration to Git repository (configs/jobs/SystemA_ETL.yaml)
 - DAG Factory (git-sync) automatically detects new/modified configuration files
 - Validates YAML schema using Pydantic, generates corresponding Airflow DAG
 - DAG becomes available in Airflow UI within 30 seconds (no manual deployment)
 - This GitOps approach ensures version-controlled, auditable workflow definitions
- **DAG Generation & Scheduling:**
 - Generated DAGs are loaded into Airflow Scheduler
 - Scheduler monitors cron schedules or event triggers (SFTP file arrival)
 - File Sensors poll SFTP/object storage for file arrival with configurable timeout and retry intervals
 - Once file detected, Scheduler activates DAG tasks in dependency order
- **File Access & Parsing (Worker Pod 1 - Stage 1):**
 - Worker pod downloads file from SFTP (or reads from shared storage) based on configuration
 - Validates file size/checksum/format; streams rows (no full file load)
 - Handles delimiter detection, encoding conversion (UTF-8/UTF-16/ANSI), header row processing
 - Applies configured skips/filters; isolates bad records below threshold
 - Writes parsed data to cache; pod terminates and releases memory
- **Transformation & Aggregation (Worker Pod 2 - Stage 2):**
 - Fresh pod reads parsed Parquet from cache
 - Applies configuration-driven transforms: trim/uppercase/type casts/calculated columns

- Runs SQL/group-by/aggregate functions via DuckDB
 - Writes processed/aggregated Parquet back to cache; pod terminates
- Database Mapping & Loading (Worker Pod 3 - Stage 3):
 - Fresh pod reads processed Parquet from cache
 - Applies CopyData mappings (source → destination fields)
 - Executes batch upserts with database-specific bulk loaders:
 - Oracle: SQL*Loader for inserts, MERGE for upserts
 - PostgreSQL: COPY for inserts, ON CONFLICT for upserts
 - Processes batches with configurable parallelism; writes completion logs to the logging store; pod terminates
- Worker Pod Execution & S3 Memory Synchronization:

This is the critical orchestration pattern for scalable, memory-efficient processing:

 - Isolated Task Execution: Scheduler spawns separate worker pods using KubernetesPodOperator for each processing stage.
 - Memory Synchronisation via object storage: Worker pods don't share memory; each stage writes results to the shared cache and terminates; the next pod reads from cache with a clean memory state.
 - Pod Lifecycle (3 pods as per diagram):
 - Worker Pod 1: File Ingestion & Parsing — watch/read SFTP, parse/validate, initial reductions, write Parquet to cache.
 - Worker Pod 2: Transformation & Aggregation — SQL/group-by/aggregates on Parquet, write processed Parquet to cache.
 - Worker Pod 3: Database Mapping & Load — read processed Parquet, apply mappings, bulk load DB, write completion logs.
 - Key Benefits:
 - No Memory Pressure: Each pod handles only one processing stage; 100M+ record datasets never fully reside in memory
 - Fault Tolerance: If a pod crashes, it restarts and re-reads from cache (idempotent operations)
 - Independent Scaling: Each task can have different CPU/memory limits optimized for its workload (e.g., aggregation: 8GB, database load: 4GB)
- External System Connections:
 - SFTP Server: Workers download files, Scheduler polls for arrival
 - Object Storage Cache: Stores intermediate Parquet results
 - Database (Oracle): Workers execute bulk upsert operations using database-specific optimizers
 - Logs: Workers emit structured logs

4.3 Technical Stack

Component	Technology	Reason for choosing this component
Language	Python 3	Excellent data processing libraries
Data Processing	DuckDB or PySpark	10-100x faster than Pandas, streaming support for files larger than RAM, multi-threaded by default, handles 100M records efficiently
Database ORM	SQLAlchemy	Connects to any database (Oracle/PostgreSQL/MySQL) with same code, manages connection pooling automatically, simplifies database operations
Orchestration	Apache Airflow	Industry-standard workflow orchestration, Python-native, rich ecosystem, supports self-hosted (Kubernetes)
Configuration	Pydantic	Pydantic for runtime type validation with clear error messages
Python Scripts	Embedded Python in YAML	Allows complex transformation logic directly in configuration files without code changes, maximum flexibility

4.4 Non-Functional Requirements Alignment

Performance

- Process large text files up to 100,000,000 records/file in minutes (not hours)
- Job trigger-to-start (scheduler/API) p95 ≤ 30s for runs

Scalability

- Container-based solution, scalable on Kubernetes
- Stateless workers; scale-out based on queue depth/files
- Support parallelism via partitioning (by file) without code changes

Availability & Resilience

- Built-in retries with backoff, and alerting on job failure
- Idempotent loads & safe re-runs (checkpointing or deterministic “exactly-once” behavior where feasible)

Security

- Secrets only via Kubernetes Secrets / vault-like mechanism (no plaintext in images/config)
- End-to-end encryption in transit (TLS) for SFTP/DB connections

Observability

- Structured logs (JSON) with correlation IDs per job/run and per source file
- Metrics for throughput, error rates, bad-record counts, retries, and per-step duration
- Alerting for failures, SLA breaches, and abnormal data quality signals

Maintainability

- Configuration-driven behaviours covering functional scope
- Complete technical + operational docs plus Docker/Helm artifacts delivered and maintained

Usage

Each milestone's Definition of Done must reference the relevant NFR bullet(s). Thresholds may be refined after load tests and production telemetry.

5 Sample Use Case Solution

This section shows, end-to-end, how the GitOps + DAG Factory + 3-pod architecture solves the RFP Section 11.2 scenario. YAML configs live in Git; DAG Factory generates the Airflow DAG; Airflow (with Kubernetes Executor) runs three isolated pods per job:

- ingest/parse SFTP file → Parquet cache
- transform/aggregate in DuckDB → Parquet cache
- map/load to DB with bulk upsert

A zip bundle (provided with the submission) contains the YAML configs for SystemA, so the client can extract and run directly.

Scenario Recap

- SystemA produces file at 1:00 AM: customers_20251205.csv
- Source file contains: RecordId, CustomerId, CustomerName, CustomerVAT, ProductCode, ProductName, Address, PhoneNumber
- Target database: Customer, Product, Address, Phone tables
- Challenge: Denormalized source file (multiple rows per customer) needs to load into normalized database tables

How the approach solves it

Config-first: YAML defines the source (SystemA_Customers with columns RecordId, CustomerId, CustomerName, CustomerVAT, ProductCode, ProductName, Address, PhoneNumber), aggregations (Customer_Unique, Address_Unique, Phone_Unique), copy mappings (CopyCustomers, CopyAddresses, CopyPhones, CopyProducts), and the job (SystemA_ETL: sensor → validate → transform/aggregate → copy tasks → post-SQL).

- GitOps + DAG Factory: Committing YAMLs to Git triggers DAG Factory to auto-generate one Airflow DAG per job; git-sync updates Airflow continuously.
- Airflow orchestration: Scheduler enforces dependencies/retries; Kubernetes Executor runs each task as its own pod.
- Three pod stages per run:
 - Ingestion & Parsing pod: read SFTP file, validate/parse, write Parquet to object-storage cache.
 - Transformation & Aggregation pod: apply configured transforms and group-bys (DuckDB/Polars), write processed Parquet to cache.
 - Database Mapping & Load pod: read processed Parquet, apply mappings, bulk upsert into the database, log results.
- Post-SQL (optional): final SQL task updates metadata/counters after loads complete.
- Observability: Logs/metrics/alerts per task; retries/backoff; bad records isolated under thresholds.

Please [click on this ZIP file](#) to access all YAML configurations for SystemA.

6 Project Execution Plan

6.1 Implementation Phases

The project is structured into five focused phases with clear deliverables and milestones to ensure systematic development, thorough testing, and smooth production deployment support.



Project execution plan can be referred from the below table:

Phase	Activities/Deliverables	Purpose	Timeline
1			
Requirement Gathering, Analysis and Architecture Design	Requirement walkthroughs	Establish a clear, agreed foundation for scope, architecture, data model, and execution approach	2 Weeks
	Clarification sessions		
	Scope validation		
	Architecture design		
	Data flow definition		
	Configuration framework design		
	Risk identification		
2			
Core ETL Engine Development	2.1 (Ingestion & Configuration Foundation) - Configuration framework setup - File ingestion engine (SFTP/S3) - Parsing engine (delimited & fixed length) - Validation & error handling	Build the ingestion and configuration backbone of the ETL platform	3 Weeks
	2.2 (Transformation & Data Processing Engine) - Transformation engine - Aggregation logic - Calculated fields - Business rules - Schema mapping, database loader (Oracle/Postgres/MySQL)	Enable core data transformation, normalization, and high-performance loading	4 Weeks
	2.3 (Orchestration and monitoring) - Job orchestration (Airflow) - Logging & monitoring	Make the ETL engine ready for testing with	2 Weeks

		observability and resilience	
3			
Integration and Testing	Retry logic	Validate correctness, performance, and readiness for production use	2 Weeks
	Performance tuning		
	End-to-end pipeline integration		
	Functional and regression testing		
	Data validation		
	Performance testing		
	Production job configuration		
	System integration testing		
	User acceptance testing		
	Performance benchmarking (100M records)		
Security validation			
4			
Deployment Support and Stabilization	Deployment readiness checks	Ensure smooth go-live and stable production execution	1 Week
	Containerized deployment		
	Helm artifact finalization		
	Production cutover assistance		
	Initial production monitoring		
	Issue resolution		
5			
Training, Documentation and Handover	Knowledge transfer sessions	Enable EXUS teams to operate independently.	1 Week
	Hands-on operational training		
	Technical and operational documentation		
	Project closure sign-off		
TOTAL			15 Weeks

6.2 Team Allocation

6.2.1 Role-wise Allocation

Role	No. of Resources	Key Responsibilities
Project Manager	1	Project planning and execution
		Requirement coordination with EXUS
		Timeline and Milestone Tracking
		Risk and change management
		Client communication and reporting
		Ensuring alignment between business expectations and technical delivery
Data Engineer	2	Design and development of the ETL engine
		File ingestion logic
		Transformation and aggregation pipelines
		Performance optimization
		Database mapping and loading
		Configuration framework implementation
DevOps Engineer	1	Containerization using Docker
		Kubernetes readiness
		Helm chart preparation
		CI/CD pipeline support
		Environment configuration

		Deployment documentation
QA Engineer	1	Test planning and execution
		Validation of data accuracy
		Functional and regression testing of ETL workflows
		Performance and data integrity testing
		Defect tracking and coordination of UAT support

6.2.3 Resource Details

Role	Name	Experience	Resume
Project Manager	Mr. Yash Patadiya	5+ years	Click here to view resume
Data Engineer	Mr. Jay Dhakan	4+ years	Click here to view resume
Data Engineer	Mr. Darshit Mehta	4+ years	Click here to view resume
QA Engineer	Mr. Kaushal Nanda	4+ years	Click here to view resume
Devops Engineer	Mr. Rushil Koyani	5+ years	Click here to view resume

6.3 Execution Methodology

6.3.1 Milestone Execution Methodology

01

Scope Understanding

- Vision & Goals
- Feature discussion
- Questions
- Mind map

Frequency:

One Time

Involvement:

☒ Kevit ☒ Exus**Outcome:**

A unified understanding of the project vision, key features, and the initial scope artifact (mind map), which will be shared with both teams.

02

Scope Validation

- Feature Listing
- Review scope
- Freeze the scope

Frequency:

One Time

Involvement:

☒ Kevit ☒ Exus**Outcome:**

An approved feature list documented in ClickUp and an official Scope Freeze email confirming the agreed-upon scope for development.

03

Sprint Planning

- Prepare backlog
- Prioritisation
- Workflow & User stories
- Test scenarios
- Design & Planning

Frequency:

Bi weekly

Involvement:

☒ Kevit**Outcome:**

Sprint backlog configured in ClickUp with all user stories, defined acceptance criteria, and test scenarios.

04

Daily Scrum

- Task allocation with due date
- Set the priority
- Blockers to resolve
- Bug/Detect resolution

Frequency:

Daily

Involvement:

☒ Kevit**Outcome:**

Daily stand-up notes are shared in Teams, and task statuses are updated in ClickUp.

05**Sprint Review Meeting** (Next sprint planning input)

- Sprint goal recap
- Demonstration of sprint
- Stakeholder feedback
- Action items and Next steps

Frequency:

Bi weekly

Involvement:

☒ Kevit ☒ Exus
Outcome:

Sprint outcomes reviewed, feedback aligned, backlog refined, and clear next steps defined.

06**SIT (System Integration Testing)**

- SIT test cases
- Validation of business scenario
- Defect management
- Retesting and defect fixes

UAT (User Acceptance Testing)

- Prepare test scenarios
- UAT environment configuration
- UAT guide document
- Defect reporting and fixes
- UAT signoff

Frequency:

Feature completion

Involvement:

☒ Kevit ☒ Exus
Outcome:

Defect logging, bug fixes and UAT signoff.

07**Go Live**

- Go-Live readiness checklist
- Deployment planning
- Communication plan
- Backup & rollback preparation
- Production testing
- Go-Live execution
- Support initiation

Frequency:

Milestone completion

Involvement:

☒ Kevit
Outcome:

Handover, documentation and training.

Our Productivity and Collaboration Tools

Planning, task tracking,
and project execution



Microsoft Teams

Team communication and
online meetings



Outlook

Email and scheduling

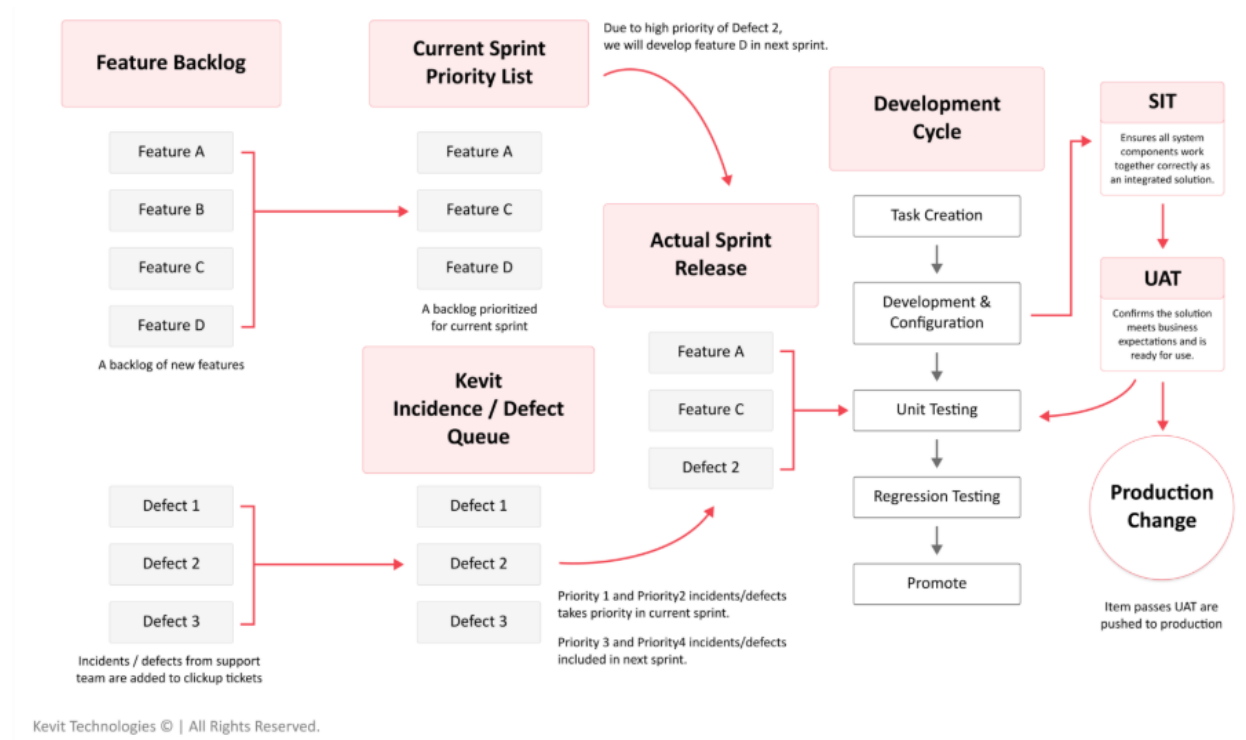


Zoho Desk

Customer support and
ticket management

6.3.2 Lifecycle of features

At Kevit, we follow a structured and transparent feature and defect lifecycle to ensure the timely and efficient delivery of high-quality software. The diagram outlines how we prioritize, develop, and release both new features and issue fixes in an agile environment.



6.3.3 Responsibility Matrix

Responsibilities	Exus	Kevit
Initial Requirements Review & Clarification	50%	50%
Detailed Scope Definition & Finalization	25%	75%
Functional Prioritization & Roadmap Planning	25%	75%
Solution Architecture & Technical Design	0%	100%
Testing (Unit Testing Integration Testing SIT UAT)	25%	75%
Training & Knowledge Transfer to EXUS Team	0%	100%
Deployment Readiness & Handover Support	25%	75%
Production Go-Live Support	50%	50%
Post-Go-Live Monitoring & Stabilization	50%	50%

Kevit Technologies © | All Rights Reserved.

6.3.4 Testing and QA Approach

Phase	Testing Activity	Description
Milestone Planning	Requirements Review & Test Scenario Definition	When: At the start of each milestone after requirements, ETL configurations, data mappings, and transformation rules are defined.
		Approach: High-level data-driven test scenarios are outlined to serve as blueprints for detailed test cases.
		Outcome: Defines data quality, correctness, and performance expectations for the milestone.
During Each Sprint	Unit Testing (Continuous Integration)	Frequency: Every code commit.
		Approach: Developers write and execute unit tests for ETL components, transformations, and configuration parsers using frameworks such as PyTest, integrated with CI/CD pipelines.
		Outcome: Immediate feedback and early resolution of parsing, transformation, and logic issues.
	Integration Testing	Frequency: At least once per development milestone.
		Approach: QA engineers validate that ingestion, transformation, aggregation, and loading components work seamlessly with existing pipelines.
		Outcome: Early identification and resolution of data flow, dependency, and schema integration issues.
End of Sprint	Regression & Validation Testing (UAT)	Frequency: At the end of every milestone.
		Approach: Automated regression suite executed via CI/CD; QA performs validation and optionally reviews with EXUS to confirm data accuracy and operational readiness.
		Outcome: Feedback on data correctness, performance behaviour, and readiness, with issues prioritized for the next milestone.
Per Milestone / Release Cycle	Performance & Load Testing	Frequency: At least once per milestone and before major releases.
		Approach: Load simulations using tools like Locust or Apache JMeter to measure throughput, response time, and stability under peak load (including 100M+ records).
		Outcome: Validation of scalability and identification of tuning or optimization requirements.
	Security Testing & Vulnerability Scanning	Approach: Automated vulnerability scanning, validation of access controls, credential handling, and secure configurations.
		Frequency: Automated scans in CI/CD; focused reviews at least once per milestone or during major integrations.
		Outcome: Mitigation of security risks and alignment with enterprise security and compliance standards (e.g., GDPR).
Final Pre-Launch Phase	Comprehensive End-to-End Testing	Frequency: Once, prior to final UAT sign-off.
		Approach: Execute full ETL cycle including ingestion, transformation, aggregation, and loading; re-run regression, performance, and security tests.
		Outcome: Stable, fully integrated ETL platform meeting all functional and non-functional requirements.
	UAT with EXUS Stakeholders	Frequency: Final stage of the project.
		Approach: UAT with production-like datasets and RFP-defined use cases; validate data accuracy, job reliability, observability, and operational workflows with EXUS stakeholders.
		Outcome: Formal confirmation of production readiness and business acceptance.

6.3.5 Team Availability

To ensure seamless collaboration and effective communication with the EXUS team, Kevit will align its working hours to maximize overlap with EXUS's operating time zone.

Working Window

08:00 AM – 06:00 PM Greece Time (EET/EEST)

(11:30 AM – 09:30 PM IST)

This overlapping window enables real-time collaboration daily, allowing for timely discussions, reviews, and decision-making without delays.

Collaboration & Communication Model

- Key collaboration activities such as daily syncs, milestone discussions, requirement clarifications, and issue resolutions will be scheduled within this overlapping time window.
- Regular touchpoints (stand-ups or status syncs, as applicable) will be conducted during this period to ensure alignment on progress, risks, and next steps.
- Outside of real-time meetings, the team will provide structured daily progress updates via email and agreed communication channels, covering:
 - Work completed
 - Work planned
 - Blockers or dependencies (if any)

6.4 Change Management Process

6.4.1 Roles & Responsibilities

Role	Responsibilities
EXUS Stakeholders	Identify and request changes related to business rules, data sources, transformations, or workflows
Product / Business Owner (EXUS)	Validates business relevance, priority, and necessity of the change
Project Manager (Kevit)	Owns the change process, performs impact assessment, communicates decisions, and tracks changes
Technical Team (Kevit)	Provides effort estimation, risk analysis, and implements approved changes

6.4.2 Change Request Submission

All change requests will be submitted using a standard Change Request format, capturing:

- Change description and rationale
- Affected layer (ingestion, transformation, aggregation, orchestration, configuration)
- Priority (Critical / High / Medium / Low)
- Expected business outcome
- Supporting details or examples (sample files, schemas, configs)

6.4.3 Initial Review & Impact Assessment

- The Project Manager reviews the request for clarity and completeness
- The technical team performs an impact assessment covering:
 - Scope impact
 - Effort estimation
 - Timeline impact
 - Performance or data integrity risks
 - Configuration vs code change dependency

Note: As part of the impact assessment, the Project Manager will also provide an effort estimate. If the change falls outside the agreed scope, the additional effort will be communicated and charged separately at a pre-agreed hourly rate, subject to EXUS approval before execution.

6.4.4 Approval & Prioritization

- Approved changes are classified as:
 - Configuration-only changes (low impact)
 - Code or architecture changes (medium/high impact)
- Changes are prioritized jointly by EXUS and the Project Manager
- Critical changes may be taken up immediately; others are scheduled into the project plan or a subsequent phase

6.4.5 Implementation & Validation

- Approved changes are implemented following existing development and testing standards
- Configuration changes undergo validation and test runs
- Code changes include unit, integration, and regression testing
- Changes are reviewed with EXUS before final acceptance

6.4.6 Documentation & Traceability

- All changes are documented and version-controlled
- Configuration updates are tracked via Git
- Impacted components and environments are recorded for audit and reference

6.5 Training and Knowledge Transfer Plan

The Training and Knowledge Transfer plan ensures the EXUS team can independently operate, maintain, and extend the ETL platform after project completion.

The goal is not only to transfer system knowledge, but to build operational confidence so EXUS can onboard new data sources, modify transformations, monitor jobs, and troubleshoot issues without vendor dependency.

Component	Inclusion	Outcome for Exus
Live Hands-on Training (Post Deployment)	Architecture and execution walkthroughs, real-world job configuration, execution, error handling, guided hands-on practice, and interactive Q&A	Confident day-to-day operations and independent issue resolution
Recorded Training & Reference Assets	On-demand recordings covering architecture, configuration-driven processing, onboarding new sources, performance tuning, scaling, and security	Faster onboarding and sustained knowledge retention
Comprehensive Documentation	Architecture diagrams, configuration and usage guides, orchestration and scheduling details, troubleshooting playbooks, and security best practices	Full operational autonomy and safe system extensibility

All documentation will be delivered in formats suitable for EXUS internal repositories and version controlled where applicable.

6.6 Risk Management

This section identifies potential project risks, their impact, probability, and mitigation strategies to ensure proactive risk management throughout the project lifecycle.

6.6.1 Risk Assessment Scale

1 = Low, 2 = Medium, and 3 = High for both Probability and Impact; Risk Score is calculated as Probability × Impact and classified as Low (1–3), Medium (4–6), or High (7–9).

Probability Level	Impact Level	Risk Score (P × I)
Low	Low	1
Low	Medium	2
Low	High	3
Medium	Low	2
Medium	Medium	4
Medium	High	6
High	Low	3
High	Medium	6
High	High	9

6.6.2 Risk Register

Risk	Risk Score	Preventive Actions	Contingency Actions	Monitoring Frequency
1				
Performance Requirements Not Met (100M Records)	6	Early performance testing; parallel architecture; high-performance libraries; benchmark validation	Performance tuning; vertical scaling; code optimization; specialist support if required	Weekly
2				
Scope Creep (Additional data sources or features requested beyond RFP scope)	6	Requirements sign-off; formal change control; phased delivery	Change requests; defer non-critical scope; timeline buffer; escalation	Weekly
3				
Integration Issues (Unexpected file formats, encoding issues, or SFTP connection failures from source systems)	4	Early file specs; sample files; validation rules; encoding detection	Flexible parsers; manual workflows; source coordination; fallback file transfer	Daily
4				
Database Performance Degradation	3	Non-prod load testing; indexing strategy; bulk loads; off-peak execution	Read replicas; partitioning; incremental loads; dedicated ETL DB if needed	Monitoring during UAT.
5				
Technology Stack Issues (Late discovery of critical bugs or version incompatibilities in Airflow or DuckDB)	2	Stable versions; early PoC; issue tracking; continuous testing	Library fallback; patch or fork; architecture adjustments	Daily